

Punto 3.1 - Fundamentos de LINQ to Entities

Práctica

Ejercicio: Explorar los fundamentos de LINQ to Entities en el proyecto AceriaData. Crear un caso de uso que ejecute consultas LINQ con diferentes operadores, analizar el SQL generado con ToQueryString y comparar el comportamiento de IEnumerable e IQueryable.

Contexto del proyecto: En el Módulo 2 se configuró la arquitectura limpia con cuatro proyectos y se aplicaron los patrones Repositorio y Unidad de Trabajo. En este punto se exploran los fundamentos de LINQ sobre el proyecto AceriaData. Esta exploración se usará en el punto 3.2 para las consultas básicas con filtros y ordenaciones.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir un caso de uso para consultas LINQ

Crear el archivo src/AceriaData.Application/UseCases/ConsultasLinqUseCase.cs:

```
using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Application.UseCases;

public class ConsultasLinqUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public ConsultasLinqUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== FUNDAMENTOS DE LINQ TO ENTITIES ===");

        DemostrarIEnumerableVsIQueryable();
        DemostrarEjecucionDiferida();
        DemostrarMaterializacion();
        DemostrarToQueryString();
    }

    private void DemostrarIEnumerableVsIQueryable()
    {
        Console.WriteLine("\n-- IEnumerable vs IQueryable --");

        // IEnumerable: carga todo en memoria y filtra en C#
        var ordenesEnumerable = _unidad.Ordenes.ObtenerTodas()
            .Where(o => o.Cliente == "Constructora del Norte")
            .ToList();

        // IQueryable: filtra en SQL
        var ordenesQueryable = _unidad.Ordenes.ObtenerQueryable()
            .Where(o => o.Cliente == "Constructora del Norte")
            .ToList();

        Console.WriteLine($"IEnumerable: {ordenesEnumerable.Count} órdenes");
    }
}
```

```

        Console.WriteLine($"Queryable: {ordenesQueryable.Count} órdenes");
    }

    private void DemostrarEjecucionDiferida()
    {
        Console.WriteLine("\n--- Ejecución diferida ---");

        var consulta = _unidad.Ordenes.ObtenerQueryable();

        Console.WriteLine("Consulta construida, sin ejecutar.");

        var cliente = "Constructora del Norte";
        if (!string.IsNullOrEmpty(cliente))
        {
            consulta = consulta.Where(o => o.Cliente == cliente);
        }

        Console.WriteLine("Filtro añadido, sin ejecutar.");

        var resultados = consulta.ToList();

        Console.WriteLine($"Consulta materializada: {resultados.Count} órdenes");
    }

    private void DemostrarMaterializacion()
    {
        Console.WriteLine("\n--- Materialización ---");

        var consulta = _unidad.Ordenes.ObtenerQueryable();

        var lista = consulta.ToList();
        Console.WriteLine($"ToList: {lista.Count} órdenes");

        var primera = consulta.FirstOrDefault();
        Console.WriteLine($"FirstOrDefault: {primera?.NumeroOrden}");

        var total = consulta.Count();
        Console.WriteLine($"Count: {total} órdenes");

        var existe = consulta.Any();
        Console.WriteLine($"Any: {existe}");
    }

    private void DemostrarToQueryString()
    {
        Console.WriteLine("\n--- ToQueryString ---");

        var consulta = _unidad.Ordenes.ObtenerQueryable()
            .Where(o => o.Cliente == "Constructora del Norte")
            .OrderBy(o => o.FechaCreacion);

        var sql = consulta.ToQueryString();
        Console.WriteLine(sql);
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación. Línea 2: using AceriaData.Domain.Entities; → importa las entidades del dominio. Línea 3: using Microsoft.EntityFrameworkCore; → importa EF Core para ToQueryString. Línea 5: namespace AceriaData.Application.UseCases; → espacio de nombres de los casos de uso. Línea 7: public class ConsultasLinqUseCase → declara el caso de uso. Línea 9: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo. Línea 11: public ConsultasLinqUseCase(IUnidadDeTrabajo unidad) → constructor. Línea 13: _unidad = unidad; → asigna el parámetro al campo. Línea 16: public void Ejecutar() → declara el método principal. Línea 18: Console.WriteLine("=== FUNDAMENTOS DE LINQ TO ENTITIES ==="); → muestra la cabecera. Línea 20: DemostrarIEnumerableVsIQueryable(); → llama al método de comparación. Línea 21: DemostrarEjecucionDiferida(); → llama al método de ejecución diferida. Línea 22: DemostrarMaterializacion(); → llama al método de materialización. Línea 23: DemostrarToQueryString(); → llama al método de análisis del SQL. Línea 26: private void DemostrarIEnumerableVsIQueryable() → declara el método. Línea 28: Console.WriteLine("\n---

IEnumerable vs IQueryable ---"); → muestra la cabecera. Línea 31: var ordenesEnumerable = _unidad.Ordenes.ObtenerTodas() → obtiene todas las órdenes como lista. Línea 32: .Where(o => o.Cliente == "Constructora del Norte") → filtra en memoria. Línea 33: .ToList(); → materializa la lista. Línea 36: var ordenesQueryable = _unidad.Ordenes.ObtenerQueryable() → obtiene la consulta como IQueryable. Línea 37: .Where(o => o.Cliente == "Constructora del Norte") → filtra en SQL. Línea 38: .ToList(); → materializa la consulta. Línea 40: Console.WriteLine(\$"Enumerable: {ordenesEnumerable.Count} órdenes"); → muestra el número. Línea 41: Console.WriteLine(\$"Queryable: {ordenesQueryable.Count} órdenes"); → muestra el número. Línea 44: private void DemostrarEjecucionDiferida() → declara el método. Línea 46: Console.WriteLine("\n--- Ejecución diferida ---"); → muestra la cabecera. Línea 48: var consulta = _unidad.Ordenes.ObtenerQueryable(); → crea la consulta base. Línea 50: Console.WriteLine("Consulta construida, sin ejecutar."); → mensaje. Línea 52: var cliente = "Constructora del Norte"; → declara el cliente. Línea 53: if (!string.IsNullOrEmpty(cliente)) → comprueba si el cliente no está vacío. Línea 55: consulta = consulta.Where(o => o.Cliente == cliente); → añade el filtro. Línea 58: Console.WriteLine("Filtro añadido, sin ejecutar."); → mensaje. Línea 60: var resultados = consulta.ToList(); → materializa la consulta. Línea 62: Console.WriteLine(\$"Consulta materializada: {resultados.Count} órdenes"); → muestra el número. Línea 65: private void DemostrarMaterializacion() → declara el método. Línea 67: Console.WriteLine("\n--- Materialización ---"); → muestra la cabecera. Línea 69: var consulta = _unidad.Ordenes.ObtenerQueryable(); → crea la consulta base. Línea 71: var lista = consulta.ToList(); → materializa con ToList. Línea 72: Console.WriteLine(\$"ToList: {lista.Count} órdenes"); → muestra el número. Línea 74: var primera = consulta.FirstOrDefault(); → obtiene la primera orden. Línea 75: Console.WriteLine(\$"FirstOrDefault: {primera?.NumeroOrden}"); → muestra el número de orden. Línea 77: var total = consulta.Count(); → cuenta las órdenes. Línea 78: Console.WriteLine(\$"Count: {total} órdenes"); → muestra el total. Línea 80: var existe = consulta.Any(); → comprueba si hay órdenes. Línea 81: Console.WriteLine(\$"Any: {existe}"); → muestra el resultado. Línea 84: private void DemostrarToQueryString() → declara el método. Línea 86: Console.WriteLine("\n--- ToQueryString ---"); → muestra la cabecera. Línea 88: var consulta = _unidad.Ordenes.ObtenerQueryable() → crea la consulta. Línea 89: .Where(o => o.Cliente == "Constructora del Norte") → filtra por cliente. Línea 90: .OrderBy(o => o.FechaCreacion); → ordena por fecha. Línea 92: var sql = consulta.ToQueryString(); → obtiene el SQL sin ejecutarlo. Línea 93: Console.WriteLine(sql); → imprime el SQL.

Error común: si la interfaz IOrdenRepositorio no tiene un método ObtenerQueryable, este código no compila. Se debe añadir el método a la interfaz.

▮ Paso 3: Añadir el método ObtenerQueryable a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    OrdenFabricacion? ObtenerPorId(int id);
    List<OrdenFabricacion> ObtenerTodas();
    IQueryable<OrdenFabricacion> ObtenerQueryable();
    OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Domain.Entities; → importa las entidades. Línea 3: namespace AceriaData.Application.Interfaces; → espacio de nombres. Línea 5: public interface IOrdenRepositorio → declara la interfaz. Línea 7: OrdenFabricacion? ObtenerPorId(int id); → método que obtiene por Id. Línea 8: List ObtenerTodas(); → método que obtiene todas. Línea 9: IQueryable ObtenerQueryable(); → método que devuelve IQueryable para composición de consultas. Línea 10: OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden); → método que obtiene por número. Línea 11: void Agregar(OrdenFabricacion orden); → método que agrega. Línea 12: void Eliminar(OrdenFabricacion orden); → método que elimina.

Error común: exponer IQueryable en el repositorio es un anti-patrón según el documento de evolución. Sin embargo, para el Módulo 3, que se centra en LINQ, es necesario para demostrar la composición de consultas. En el Módulo 4 se refactorizará para encapsular las consultas en métodos específicos.

| Paso 4: Implementar ObtenerQueryable en el repositorio

Modificar src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs:

```
public IQueryable<OrdenFabricacion> ObtenerQueryable()
{
    return _context.OrdenesFabricacion.AsQueryable();
}
```

Línea 1: public IQueryable ObtenerQueryable() → declara el método. Línea 3: return _context.OrdenesFabricacion.AsQueryable(); → devuelve la consulta como IQueryable.

Error común: si se devuelve _context.OrdenesFabricacion.ToList().AsQueryable(), se materializa la consulta antes de tiempo y se pierde la ventaja de la traducción a SQL.

| Paso 5: Registrar el caso de uso en el contenedor

Modificar src/AceriaData.Console/Program.cs:

```
services.AddScoped<ConsultasLinqUseCase>();
```

Línea 1: services.AddScoped(); → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 6: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<ConsultasLinqUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito. Línea 3: var useCase = scope.ServiceProvider.GetRequiredService(); → resuelve el caso de uso. Línea 4: useCase.Ejecutar(); → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción InvalidOperationException.

| Paso 7: Insertar datos de prueba

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes de prueba:

```

using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var ordenes = new List<OrdenFabricacion>
    {
        new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = DateTime.Now },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = DateTime.Now },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Norte", Estado = "EnProceso", FechaCreacion = DateTime.Now },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0004", Cliente = "Constructora del Este", Estado = "Pendiente", FechaCreacion = DateTime.Now }
    };

    context.OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();
}

```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito. Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el DbContext. Línea 4: `context.Database.EnsureDeleted();` → elimina la base. Línea 5: `context.Database.EnsureCreated();` → crea la base. Línea 7: `var ordenes = new List` → crea la lista de órdenes. Línea 9: `new OrdenFabricacion { ... }`, → primera orden. Línea 10: `new OrdenFabricacion { ... }`, → segunda orden. Línea 11: `new OrdenFabricacion { ... }`, → tercera orden. Línea 12: `new OrdenFabricacion { ... }` → cuarta orden. Línea 15: `context.OrdenesFabricacion.AddRange(ordenes);` → registra las órdenes. Línea 16: `context.SaveChanges();` → inserta las órdenes.

Error común: si se insertan las órdenes con el mismo número, la clave alternativa `AK_OrdenesFabricacion_NumeroOrden` rechaza la inserción.

Paso 8: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observa la diferencia entre `IEnumerable` e `IQueryable`, la ejecución diferida, la materialización y el SQL generado.

Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

=== FUNDAMENTOS DE LINQ TO ENTITIES ===

--- IEnumerable vs IQueryable ---

Enumerable: 2 órdenes

Queryable: 2 órdenes

--- Ejecución diferida ---

Consulta construida, sin ejecutar.

Filtro añadido, sin ejecutar.

Consulta materializada: 2 órdenes

--- Materialización ---

ToList: 4 órdenes

FirstOrDefault: OF-2024-0001

Count: 4 órdenes

Any: True

--- ToQueryString ---

SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado]

FROM [OrdenesFabricacion] AS [o]

WHERE [o].[Cliente] = N'Constructora del Norte'

ORDER BY [o].[FechaCreacion]

La primera sección muestra que ambas consultas devuelven dos órdenes, pero la primera filtra en memoria y la segunda en SQL. La segunda sección muestra que la consulta no se ejecuta hasta ToList. La tercera sección muestra los diferentes métodos de materialización. La cuarta sección muestra el SQL generado.

Observaciones: el SQL generado por ToQueryString no se ha ejecutado contra la base de datos. Solo se ha construido. Esto demuestra que ToQueryString no materializa la consulta.

Paso 10: Diagnosticar un error común

Modificar el método DemostrarIEnumerableVsIQueryable para usar ToList antes del filtro:

csharp

```
var ordenesEnumerable = _unidad.Ordenes.ObtenerTodas()
    .ToList()
    .Where(o => o.Cliente == "Constructora del Norte")
    .ToList();
```

Resultado esperado: el resultado es el mismo, pero la consulta SQL ejecutada carga todas las órdenes en memoria y después filtra. Si la tabla tuviera un millón de filas, la consulta sería muy lenta.

Solución: aplicar ToList solo al final, después de todos los filtros.

Errores comunes del ejercicio Error Causa Solución Filtro se aplica en memoria Se llamó a ToList antes del filtro Aplicar ToList al final Expresión no traducida Se usó un método personalizado Reescribir con métodos traducibles Múltiples consultas Se materializó varias veces Materializar una sola vez al final ToQueryString ejecuta la consulta No, solo la construye Verificar que no se llama a ToList IQueryable expuesto en el repositorio Anti-patrón Encapsular en métodos específicos (Módulo 4) Falta el using de EF Core El método ToQueryString no está disponible Añadir using Microsoft.EntityFrameworkCore;

Retos resueltos: Analizar el SQL de una consulta con filtros y ordenaciones

Reto: Crear un método que construya una consulta LINQ con filtros por cliente y estado, ordenación por fecha, y proyección a un tipo anónimo. Obtener el SQL con ToQueryString y analizar su estructura.

Solución paso a paso:

Paso 1: Añadir el método AnalizarSql al caso de uso:

```
private void AnalizarSql()
{
    Console.WriteLine("\n--- Análisis del SQL generado ---");

    var consulta = _unidad Ordenes.ObtenerQueryable()
        .Where(o => o.Cliente == "Constructora del Norte" && o.Estado == "Pendiente")
        .OrderByDescending(o => o.FechaCreacion)
        .Select(o => new { o.NumeroOrden, o.Cliente, o.Estado, o.FechaCreacion });

    var sql = consulta.ToQueryString();
    Console.WriteLine(sql);
}
```

Línea 1: `private void AnalizarSql()` → declara el método. Línea 3: `Console.WriteLine("\n--- Análisis del SQL generado ---");` → muestra la cabecera. Línea 5: `var consulta = _unidad.Ordenes.ObtenerQueryable()` → inicia la consulta. Línea 6: `.Where(o => o.Cliente == "Constructora del Norte" && o.Estado == "Pendiente")` → filtra por cliente y estado. Línea 7: `.OrderByDescending(o => o.FechaCreacion)` → ordena por fecha descendente. Línea 8: `.Select(o => new { o.NumeroOrden, o.Cliente, o.Estado, o.FechaCreacion })`; → proyecta a un tipo anónimo. Línea 10: `var sql = consulta.ToQueryString();` → obtiene el SQL. Línea 11: `Console.WriteLine(sql);` → imprime el SQL.

Paso 2: Llamar al método desde Ejecutar:

```
AnalizarSql();
Paso 3: Ejecutar dotnet run y analizar el SQL generado.
```

Resultado esperado: el SQL incluye el filtro WHERE [o].[Cliente] = N'Constructora del Norte' AND [o].[Estado] = N'Pendiente', la ordenación ORDER BY [o].[FechaCreacion] DESC y la proyección de las columnas seleccionadas.

Analugía final

LINQ to Entities en una acería es como el lenguaje que usa el jefe de planta para pedir informes al archivo central. El jefe no va al archivo a buscar los documentos: envía una orden con las condiciones (cliente, estado, fecha) y el archivo devuelve solo los documentos que cumplen esas condiciones. Si el jefe pidiera todos los documentos y luego los filtrara él mismo, perdería tiempo y esfuerzo. Eso es lo que pasa cuando se usa `IEnumerable` en lugar de `IQueryable`: se traen todos los datos a memoria y se filtran en el cliente. La ejecución diferida es como preparar la orden de búsqueda antes de enviarla: el jefe puede añadir condiciones hasta que esté seguro, y solo entonces la envía. La materialización es el momento en que se envía la orden y se recibe la respuesta. `ToQueryString` es como pedir una copia de la orden sin enviarla: el jefe puede revisar qué se va a pedir antes de ejecutarlo. Así funciona LINQ to Entities: se construye la consulta, se traduce a SQL y se ejecuta en el servidor, aprovechando toda la potencia de la base de datos.

Resultado esperado

Al final del ejercicio, deberías haber:

Creado el caso de uso `ConsultasLinqUseCase`.

Añadido el método `ObtenerQueryable` a la interfaz del repositorio.

Implementado el método en el repositorio.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba.

Ejecutado las demostraciones de `IEnumerable` vs `IQueryable`, ejecución diferida, materialización y `ToQueryString`.

Analizado el SQL generado por una consulta con filtros y ordenaciones.

Diagnosticado el error de aplicar ToList antes del filtro.

Creado el método AnalizarSql.
